

claude-windows / 73a3938e-1da3-4e28-bfa7-346a15718301

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301\subagents\workflows\wf_753ff171-b1d\agent-a687cc70ecfd80619.jsonl`  
- SHA-256: `6767ffe280d9e108c89bdf614d643914cf851c55fa8631a6093743e497489c40`  
- Source modified: `2026-06-22T12:36:39+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_753ff171-b1d`  
- session_id: `73a3938e-1da3-4e28-bfa7-346a15718301`
```

Transcript

1. user (2026-06-22T12:34:19.537Z)

You are checking that the NEW tests in PR #319 of the local git repo (Khelsutra/badminton-highlight-indexer) genuinely guard the fix and are not tautological/weak.

Read the tests: ``git show origin/fix/worker-train-dated-label-join:tests/test_worker.py`` (focus on the `cmd_train / _label_clip_name` tests added vs master: ``git diff origin/master..origin/fix/worker-train-dated-label-join -- tests/test_worker.py``).

Verify with concrete reasoning:

1. ``test_label_clip_name_strips_date_prefix`` covers the meaningful cases (dated, legacy bare, `_proxy` kept, underscores-in-name survive, bare .csv) and asserts the RIGHT mappings.
2. ``test_cmd_train_detector_joins_dated_labels_to_bare_videos`` actually exercises the dated?bare join through ``cmd_train`` (not a mocked-away no-op) and would FAIL on master (where the helper doesn't exist / the date isn't stripped). Explain the mechanism by which it fails pre-fix.
3. The skip-branch tests (no-matching-video, unprobeable fps, no-trajectory, <2-videos abort) assert observable outcomes (rc and/or manifest spec names), not just "doesn't crash".
4. No assertion is trivially always-true; monkeypatches don't accidentally stub out the very code under test.

You MAY run targeted checks (e.g. ``git show`` to inspect, or reason about monkeypatch targets). Return a verdict; ok=false if a test is tautological or doesn't actually guard the regression.

2. assistant (2026-06-22T12:34:24.078Z)

I'll analyze the new tests in PR #319 to verify they genuinely guard the fix. Let me start by examining the diff and the relevant code.

3. user (2026-06-22T12:34:27.055Z)

From <https://github.com/Khelsutra/badminton-highlight-indexer>

```
* branch          fix/worker-train-dated-label-join -> FETCH_HEAD
```

```
---DIFF---
```

```
diff --git a/tests/test_worker.py b/tests/test_worker.py
```

```
index 642e46a..070ac64 100644
```

```
--- a/tests/test_worker.py
```

```
+++ b/tests/test_worker.py
```

```
@@ -191,6 +191,196 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
```

```
# --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---
```

```
+@pytest.mark.parametrize("fname,expected", [
```

```
+    ("2026-06-22_GX020094.rallies.csv", "GX020094"),          # canonical dated label
```

```
(DATA_IN_GCS $7.1)
```

```
+    ("GX020094.rallies.csv", "GX020094"),                    # legacy bare label -> SAME
```

```
clip stem
```

```

+ ("GX020094_proxy.rallies.csv", "GX020094_proxy"),          # _proxy is part of the name,
kept
+ ("2026-06-21_mahadevpura_1.rallies.csv", "mahadevpura_1"),  # underscores in <name> survive
+ ("v1.csv", "v1"),                                           # bare .csv suffix
+ ("2026-06-22_GX020094.csv", "GX020094"),                   # dated + bare .csv
+])
+def test_label_clip_name_strips_date_prefix(fname, expected):
+    """A dated label and a bare label must both resolve to the bare videos/<name> stem."""
+    assert wmain._label_clip_name(fname) == expected
+
+
+def test_cmd_train_detector_joins_dated_labels_to_bare_videos(tmp_path, monkeypatch):
+    """Regression (DATA_IN_GCS $7b blocker #1): canonical labels carry an ISO-date prefix
+    (labels/<date>_<name>.rallies.csv) while videos are bare (videos/<name>.mp4). The label-
+    derived
+    clip name must strip the date so the video join succeeds ? otherwise every clip is skipped
+    and
+    train aborts with <2 videos."""
+    import os
+    import subprocess
+
+    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub") # CPU stub runner (no GPU/WSL)
+    monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
+    corpus = tmp_path / "corpus"
+    labels_dir = corpus / "labels"
+    videos_dir = corpus / "videos"
+    labels_dir.mkdir(parents=True)
+    videos_dir.mkdir(parents=True)
+    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
+    # canonical DATED labels ...
+    (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(header +
+    "1,2.0,5.0,winner,badminton,\n")
+    (labels_dir / "2026-06-21_mahadevpura_1.rallies.csv").write_text(header +
+    "1,3.0,6.0,winner,badminton,\n")
+    # ... joining to BARE-stem videos
+    (videos_dir / "GX020094.mp4").write_bytes(b"FAKEVIDEO1")
+    (videos_dir / "mahadevpura_1.mp4").write_bytes(b"FAKEVIDEO2")
+    out_dir = tmp_path / "out"
+    scratch = tmp_path / "scratch"
+    cfg = tmp_path / "config.json"
+    cfg.write_text("{}")
+
+    def fake_run(cmd, *a, **k):
+        out = cmd[cmd.index("--out") + 1]
+        ver = cmd[cmd.index("--version") + 1]
+        bdir = os.path.join(out, ver)
+        os.makedirs(bdir, exist_ok=True)
+        for fn in ("weights.json", "manifest.json"):
+            with open(os.path.join(bdir, fn), "w") as f:
+                f.write("{}")
+        return subprocess.CompletedProcess(cmd, 0)
+    monkeypatch.setattr(subprocess, "run", fake_run)
+
+    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
+    "--version", "0.0.4", "--config", str(cfg), "--scratch", str(scratch),
+    "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
+    assert rc == 0 # before the fix: every clip skipped -> rc 2 ("<2 videos")
+    specs = json.loads((scratch / "manifest.json").read_text())
+    assert len(specs) == 2 # both dated labels joined
+    assert {s["name"] for s in specs} == {"GX020094", "mahadevpura_1"} # downstream name is
+    date-stripped
+
+
+def test_cmd_train_detector_skips_label_with_no_matching_video(tmp_path, monkeypatch):
+    """A dated label whose <name> has no videos/<name>.ext is SKIPPED (the join-miss branch),

```

```

+ while the labels that DO join are kept ? so one orphan label can't abort an otherwise-valid
run."""
+ import os
+ import subprocess
+
+ monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")
+ monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
+ corpus = tmp_path / "corpus"
+ labels_dir = corpus / "labels"
+ videos_dir = corpus / "videos"
+ labels_dir.mkdir(parents=True)
+ videos_dir.mkdir(parents=True)
+ header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
+ row = "1,2.0,5.0,winner,badminton,\n"
+ (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(header + row)
+ (labels_dir / "2026-06-21_mahadevpura_1.rallies.csv").write_text(header + row)
+ (labels_dir / "2026-06-20_orphan.rallies.csv").write_text(header + row) # NO
videos/orphan.*
+ (videos_dir / "GX020094.mp4").write_bytes(b"FAKEVIDEO1")
+ (videos_dir / "mahadevpura_1.mp4").write_bytes(b"FAKEVIDEO2")
+ out_dir = tmp_path / "out"
+ scratch = tmp_path / "scratch"
+ cfg = tmp_path / "config.json"
+ cfg.write_text("{}")
+
+ def fake_run(cmd, *a, **k):
+     out = cmd[cmd.index("--out") + 1]
+     ver = cmd[cmd.index("--version") + 1]
+     bdir = os.path.join(out, ver)
+     os.makedirs(bdir, exist_ok=True)
+     for fn in ("weights.json", "manifest.json"):
+         with open(os.path.join(bdir, fn), "w") as f:
+             f.write("{}")
+     return subprocess.CompletedProcess(cmd, 0)
+ monkeypatch.setattr(subprocess, "run", fake_run)
+
+ rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
+                 "--version", "0.0.5", "--config", str(cfg), "--scratch", str(scratch),
+                 "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
+ assert rc == 0 # the orphan label was skipped; the two joinable clips carried the run
+ specs = json.loads((scratch / "manifest.json").read_text())
+ assert {s["name"] for s in specs} == {"GX020094", "mahadevpura_1"} # 'orphan' absent
+
+
+def test_cmd_train_detector_skips_unprobeable_and_trajless_videos(tmp_path, monkeypatch):
+    """cmd_train's detector loop must SKIP (never crash, never guess) a video whose fps/width
won't
+ probe and a video the detector yields no trajectory for, keeping the healthy clips."""
+ import os
+ import subprocess
+
+ corpus = tmp_path / "corpus"
+ labels_dir = corpus / "labels"
+ videos_dir = corpus / "videos"
+ labels_dir.mkdir(parents=True)
+ videos_dir.mkdir(parents=True)
+ header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
+ row = "1,2.0,5.0,winner,badminton,\n"
+ for stem in ("good1", "good2", "badfps", "notraj"):
+     (labels_dir / f"2026-06-22_{stem}.rallies.csv").write_text(header + row)
+     (videos_dir / f"{stem}.mp4").write_bytes(b"FAKE-" + stem.encode())
+
+ # probe fails ONLY for badfps -> that clip is skipped (a wrong fps mis-aligns every label).
+ monkeypatch.setattr(wmain, "_probe_video_fps_width",
+                     lambda p: None if "badfps" in os.path.basename(p) else (30.0, 1280.0))

```

```

+ monkeypatch.setattr(wmain, "compute_video_id", lambda p: "id-" + os.path.basename(p))
+
+ class _FakeRunner:
+     # writes a real trajectory for everything EXCEPT notraj (empty result -> skipped).
+     def run_predict(self, video, out_dir, video_id=None):
+         if "notraj" in os.path.basename(video):
+             return ""
+         traj = os.path.join(out_dir, f"{video_id}_traj.csv")
+         with open(traj, "w") as f:
+             f.write("frame,x,y\n0,1,2\n")
+         return traj
+ monkeypatch.setattr(wmain, "_resolve_train_detector", lambda args, config: _FakeRunner())
+
+ out_dir = tmp_path / "out"
+ scratch = tmp_path / "scratch"
+ cfg = tmp_path / "config.json"
+ cfg.write_text("{}")
+
+ def fake_run(cmd, *a, **k):
+     out = cmd[cmd.index("--out") + 1]
+     ver = cmd[cmd.index("--version") + 1]
+     bdir = os.path.join(out, ver)
+     os.makedirs(bdir, exist_ok=True)
+     for fn in ("weights.json", "manifest.json"):
+         with open(os.path.join(bdir, fn), "w") as f:
+             f.write("{}")
+     return subprocess.CompletedProcess(cmd, 0)
+ monkeypatch.setattr(subprocess, "run", fake_run)
+
+ rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
+                 "--version", "0.0.6", "--config", str(cfg), "--scratch", str(scratch),
+                 "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
+ assert rc == 0
+ specs = json.loads((scratch / "manifest.json").read_text())
+ assert {s["name"] for s in specs} == {"good1", "good2"} # badfps + notraj skipped
+
+
+def test_cmd_train_detector_aborts_clean_when_fewer_than_two_videos_join(tmp_path,
monkeypatch):
+    """When labels join to <2 videos, train aborts cleanly with rc 2 (leave-one-video-out needs
+>=2).
+    This is the guard the date-prefix bug used to trip for EVERY clip (DATA_IN_GCS $7b #1);
+here it
+    fires legitimately because only one of two labels has a matching video."""
+    import subprocess
+
+    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")
+    monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
+    corpus = tmp_path / "corpus"
+    labels_dir = corpus / "labels"
+    videos_dir = corpus / "videos"
+    labels_dir.mkdir(parents=True)
+    videos_dir.mkdir(parents=True)
+    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
+    row = "1,2.0,5.0,winner,badminton,\n"
+    (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(header + row) # joins
+    (labels_dir / "2026-06-21_orphan.rallies.csv").write_text(header + row) # no video ->
skipped
+    (videos_dir / "GX020094.mp4").write_bytes(b"FAKEVIDEO1")
+    cfg = tmp_path / "config.json"
+    cfg.write_text("{}")
+    # subprocess.run must NOT be reached (we abort before the harness); guard that.
+    monkeypatch.setattr(subprocess, "run", lambda *a, **k: pytest.fail("harness ran despite <2
videos"))
+

```

```

+ rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(tmp_path / "out"),
+                 "--version", "0.0.7", "--config", str(cfg), "--scratch", str(tmp_path /
+ "s"),
+                 "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
+ assert rc == 2 # only 1 of 2 labels joined -> below the leave-one-video-out floor
+
+
+ def test_cmd_train_orchestration(tmp_path, monkeypatch):
+     import os
+     import subprocess

```

4. assistant (2026-06-22T12:34:29.572Z)

Now let me look at the actual fix in the worker source to verify the tests guard the right code. Let me see the source-side diff and the helper implementation.

5. user (2026-06-22T12:34:31.440Z)

```

===SOURCE DIFF (backend/worker)===
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 081d8f5..08d8d84 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -20,6 +20,7 @@ import csv
 import json
 import logging
 import os
+import re
 import sys
 import tempfile
 from typing import Any, List
@@ -264,6 +265,29 @@ def cmd_infer(args: argparse.Namespace) -> int:
     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem collision).
     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

+    #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-DD>_<name>.rallies.csv`,
+    #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date but is
+    #: NOT
+    #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
+    _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
+
+    +
+    +
+    + def _label_clip_name(fname: str) -> str:
+    +     """Derive the clip name (the `videos/<name>.<ext>` join key) from a label filename.
+    +
+    +     Strips both the `rallies.csv` / `.csv` suffix and an optional leading ISO-date prefix,
+    +     so a
+    +     canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
+    +
+    +     * `2026-06-22_GX020094.rallies.csv` -> `GX020094` (canonical, date-stamped)
+    +     * `GX020094.rallies.csv` -> `GX020094` (legacy bare ? same stem)
+    +     * `GX020094_proxy.rallies.csv` -> `GX020094_proxy` (`_proxy` is part of the
+    +     name, kept)
+    +     * `2026-06-21_mahadevpura_1.rallies.csv` -> `mahadevpura_1` (underscores in <name>
+    +     survive)
+    +
+    +     Without the date strip, `--trajectory-source detector` can't match a dated label to its
+    +     bare
+    +     video stem, so every clip is skipped and train aborts with `<2 videos` (DATA_IN_GCS §7b
+    +     #1).
+    +     """
+    +     name = fname.replace(".rallies.csv", "").replace(".csv", "")
+    +     return _LABEL_DATE_PREFIX.sub("", name)
+
+

```

```

def _probe_video_fps_width(path: str):
    """Robust ffmpeg (fps, frame_width) for a real video. Returns None on ANY failure.
@@ -380,8 +404,8 @@ def cmd_train(args: argparse.Namespace) -> int:
    print(f"[worker] trajectory source: {args.trajectory_source}")
    specs: List[dict] = []
    for uri in label_uris:
-         fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
-         name = fname.replace(".rallies.csv", "").replace(".csv", "")
+         fname = storage.parse_uri(uri).name # e.g.
2026-06-22_GX020094.rallies.csv
+         name = _label_clip_name(fname) # -> GX020094 (date prefix
stripped)
        local_label = storage.fetch(uri, os.path.join(labels_dir, fname), config=storage_cfg)
        if args.trajectory_source == "detector":
            vuri = video_by_stem.get(name)
===FILE LIST CHANGED===
backend/worker/__main__.py
deploy/gcp/nightly_regression/README.md
deploy/gcp/nightly_regression/cloudbuild.launcher.yaml
deploy/gcp/nightly_regression/launch.sh
deploy/gcp/nightly_regression/register_scheduler.sh
deploy/gcp/nightly_regression/setup_email.sh
docs/DATA_IN_GCS.md
docs/README.md
tests/test_worker.py

```

6. assistant (2026-06-22T12:34:35.790Z)

The fix is clear. Now let me read the full `cmd\_train` function on the fix branch to understand the control flow that the tests exercise (the skip branches, the `<2 videos` abort, the manifest writing).

7. user (2026-06-22T12:34:36.867Z)

```

        print(f"[worker] detector environment not ready: {msg}")
        return None
    print(f"[worker] detector ready ({args.segmenter}): {msg}")
    return runner

def cmd_train(args: argparse.Namespace) -> int:
    """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest -> shell out
    to training.gen0.harness (backend must NOT import training) -> push the artifact bundle.

    Two trajectory sources (the train pipeline + harness are identical either way):
    * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent synthetic
      shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box / CI.
    * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
      DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test). This
      is the M3.5 "first real training" path; only the trajectory source flips.
    """
    import subprocess

    config = load_config(args.config) if args.config else load_config()
    _apply_overrides(config, args.set)
    if not _run_startup_checks(config):
        return 2
    storage_cfg = config.storage.model_dump()

    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
    labels_dir = os.path.join(scratch, "labels")
    traj_dir = os.path.join(scratch, "trajectories")
    videos_dir = os.path.join(scratch, "videos")
    os.makedirs(labels_dir, exist_ok=True)

```

```

os.makedirs(traj_dir, exist_ok=True)

corpus = args.corpus_uri.rstrip("/")
label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
                    if u.lower().endswith(".csv"))
if len(label_uris) < 2:
    print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
        f"under {corpus}/labels/")
    return 2

# Real-detector source: resolve the runner once + index the bucket's videos/ by stem.
runner = None
video_by_stem: dict = {}
if args.trajectory_source == "detector":
    os.makedirs(videos_dir, exist_ok=True)
    runner = _resolve_train_detector(args, config)
    if runner is None:
        return 2
    for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
        vname = storage.parse_uri(vuri).name
        if not vname.lower().endswith(_VIDEO_EXTS):
            continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow the video
        video_by_stem[os.path.splitext(vname)[0]] = vuri

print(f"[worker] trajectory source: {args.trajectory_source}")
specs: List[dict] = []
for uri in label_uris:
    fname = storage.parse_uri(uri).name # e.g.
2026-06-22_GX020094.rallies.csv
    name = _label_clip_name(fname) # -> GX020094 (date prefix
stripped)
    local_label = storage.fetch(uri, os.path.join(labels_dir, fname), config=storage_cfg)
    if args.trajectory_source == "detector":
        vuri = video_by_stem.get(name)
        if not vuri:
            print(f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping.")
            continue
        local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                config=storage_cfg)
        video_id = compute_video_id(local_video)
        # Real fps/width drive the trajectory frame->time mapping; PROBE (don't guess) ? a
        # wrong fps silently mis-aligns every label, so skip the video if ffprobe can't read
it.
        meta = _probe_video_fps_width(local_video)
        if meta is None:
            print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ? skipping
(won't guess).")
            continue
        fps, frame_width = meta
        traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
        if not traj:
            print(f"[worker] detector produced no trajectory for {name!r} ? skipping.")
            continue
        specs.append({"name": name, "trajectory": traj, "labels": local_label,
                      "fps": fps, "frame_width": frame_width})
    else:
        from backend.pipeline.detectors.stub_runner import synth_trajectory_from_labels
        traj = os.path.join(traj_dir, f"{name}_traj.csv")
        synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
        specs.append({"name": name, "trajectory": traj, "labels": local_label,
                      "fps": args.fps, "frame_width": args.frame_width})

```

```

    if len(specs) < 2:
        print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got {len(specs)}.")
        return 2
    manifest_path = os.path.join(scratch, "manifest.json")
    with open(manifest_path, "w", encoding="utf-8") as f:
        json.dump(specs, f, indent=2)
    src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
    print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

    out_dir = os.path.join(scratch, "artifacts")
    cmd = [sys.executable, "-m", "training.gen0.harness",
           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
    if args.floor:
        floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
        config=storage_cfg)
        if "://" in args.floor else args.floor)
        cmd += ["--floor", floor_local]
    print("[worker] training:", " ".join(cmd))
    rc = subprocess.run(cmd).returncode
    if rc != 0:
        print(f"[worker] gen0 harness failed (rc={rc})")
        return rc

    # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
    bundle = os.path.join(out_dir, args.version)
    out_prefix = args.out_uri.rstrip("/")
    pushed = []
    for fn in sorted(os.listdir(bundle)):
        uri = f"{out_prefix}/weights/{args.version}/{fn}"
        storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
        pushed.append(uri)
    print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
    for u in pushed:
        print("    ", u)
    return 0

def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="python -m backend.worker",
                                description="Storage-aware worker: pull ? run pipeline ? push.")
    p.add_argument("-v", "--verbose", action="store_true",
                   help="DEBUG logging (the native detector command, frame counts, per-stage detail)")
    sub = p.add_subparsers(dest="cmd", required=True)

    pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
    pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// / gcs://")
    pi.add_argument("--out-uri", required=True, help="output prefix (outputs/<video_id>/? is appended)")
    pi.add_argument("--segmenter", default=None, help="default: indexing.default_segmenter")
    pi.add_argument("--config", default=None, help="config.json path (default: ./config.json)")
    pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp dir)")
    pi.add_argument("--max-frames", type=int, default=None)
    pi.add_argument("--done-uri", default=None,
                   help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
                   "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
                   "Default-OFF: omit it for the normal local run.")
    pi.add_argument("--set", action="append", default=[], metavar="k=v",
                   help="config override, e.g. --set indexing.skip_ai_handoff=true")
    pi.set_defaults(func=cmd_infer)

    pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)")

```



```

    pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+ videos/
for "
                    "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
    pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/ is
appended)")
    pt.add_argument("--version", required=True, help="semver for the artifact bundle")
    pt.add_argument("--trajectory-source", choices=["synth", "detector"], default="synth",
                    help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
                        "detector = REAL shuttle trajectories via the resolved DetectorRunner "
                        "(needs videos/ in the corpus; honours detector_impl).")
    pt.add_argument("--segmenter", default="wasb_hybrid",
                    help="trajectory hybrid whose detector is used for --trajectory-source
detector")
    pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or storage
URI)")
    pt.add_argument("--config", default=None)
    pt.add_argument("--scratch", default=None)
    pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps (detector
probes the video)")
    pt.add_argument("--frame-width", type=float, default=1280.0,
                    help="synth trajectory frame width (detector probes the video)")
    pt.add_argument("--set", action="append", default=[], metavar="k=v")
    pt.set_defaults(func=cmd_train)
    return p

```

```

def _setup_logging(verbose: bool) -> None:
    """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR diagnostics
are
    VISIBLE ? the worker previously ran with no logging config, so every segmenter/detector log

```

8. assistant (2026-06-22T12:34:41.903Z)

Now I understand the full control flow. Let me verify a few critical things about the test mechanics:

1. The `\_resolve\_train\_detector` and stub runner resolution ? to confirm the "stub" env path and `\_FakeRunner` monkeypatch are wired correctly.
2. Whether the `--out-uri` join (the manifest is written to `scratch/manifest.json`, which the test reads).
3. The `<2 videos` abort path and whether `subprocess.run` is reached.

Let me check the stub runner and the `\_resolve\_train\_detector` to confirm the "stub" path actually produces a trajectory (so the join test reaches the manifest), and check the test imports/setup at the top of the file.

9. user (2026-06-22T12:34:46.433Z)

```

===test_worker.py top (imports + wmain alias)===
"""Worker dispatcher: pull (storage) ? run pipeline ? push (storage).

```

```

`cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is validated
on the droplet against MinIO.

```

```

"""
import json

```

```

import pytest

```

```

from backend.config import load_config
from backend.worker import __main__ as wmain

```

----- pure helpers

```

@pytest.mark.parametrize("raw,expected", [
    ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid", "wasb_hybrid"),
])
def test_coerce(raw, expected):
    assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)

def test_apply_overrides_on_typed_config():
    cfg = _fresh_config()
    wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
    "indexing.min_segment_duration=1.5",
                                "sport=tennis"])
    assert cfg.indexing.skip_ai_handoff is True
    assert cfg.indexing.min_segment_duration == 1.5
    assert cfg.sport == "tennis"

def test_apply_overrides_rejects_bad_format():
    cfg = _fresh_config()
    with pytest.raises(ValueError):
        wmain._apply_overrides(cfg, ["no_equals_sign"])

def test_write_intervals_csv(tmp_path):
    segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4, "shot_count_confidence":
    "LocalNoAI"}],

===_resolve_train_detector + stub path===
def _resolve_train_detector(args: argparse.Namespace, config: Any):
    """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
    detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU box,
    stub=CPU mock). Returns the runner, or prints an error + returns None.

    Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build the
    detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo) has
    no shuttle detector and is rejected.
    """
    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

    seg_cls = segmenter_registry.get(args.segmenter)
    if not seg_cls:
        print(f"[worker] unknown segmenter: {args.segmenter!r} "
              f"(have {list(segmenter_registry.list_available())})")
        return None
    seg = seg_cls(None, config)
    if not isinstance(seg, TrajectoryHybridSegmenter):
        print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
              f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid).")
        return None
    try:
        runner, _ = seg._resolve_runner(config.get("indexing", {}))
    except NotImplementedError as e:
        # e.g. detector_impl='native' for a family without a native runner (tracknet, or
        # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw traceback.
        print(f"[worker] detector not available: {e}")
        return None
    ok, msg = runner.healthcheck()
    if not ok:
        print(f"[worker] detector environment not ready: {msg}")
        return None
    print(f"[worker] detector ready ({args.segmenter}): {msg}")
    return runner

def cmd_train(args: argparse.Namespace) -> int:

```

10. assistant (2026-06-22T12:34:50.596Z)

Now I understand the "stub" path. In the join test (``test_cmd_train_detector_joins_dated_labels_to_bare_videos``), ``RALLY_DETECTOR_IMPL=stub`` is set and ``_resolve_train_detector`` is NOT monkeypatched, so the real resolution runs and produces a stub runner. Let me confirm the stub runner's ``run_predict`` actually writes a trajectory file (so the join test reaches the manifest with 2 specs), and confirm ``_run_startup_checks`` and ``storage.list_uris`` work on a local filesystem corpus path (no ``://`` scheme).

11. user (2026-06-22T12:34:52.342Z)

```
===stub runner run_predict===
1: """CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".
3:A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
4:CSV with **no GPU, no WSL, no model weights** ? so the whole trajectory-hybrid pipeline
12: pipeline with a committed/golden trajectory; deterministic + cross-OS).
13: * **synth** (default) ? generate a deterministic synthetic trajectory of
52:     return cls(
65:     """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""
70:     def healthcheck(self) -> Tuple[bool, str]:
71:         return True, "stub CPU detector (no GPU/WSL)"
95:         return rows
97:     def run_predict(self, video_win_path: str, output_win_dir: str,
99:         """Write a deterministic trajectory CSV and return its path (never touches the
GPU/WSL)."""
107:         logger.info("Stub runner: replayed trajectory %s -> %s", self.cfg.replay_csv,
out_csv)
108:         return out_csv
112:         w = _csv.writer(f)
113:         w.writerow(["Frame", "Visibility", "X", "Y"])
115:         w.writerow([frame, vis, f"{x:.3f}", f"{y:.3f}"])
116:         logger.info("Stub runner: wrote %d synthetic trajectory rows -> %s", len(rows),
out_csv)
117:         return out_csv
120:     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
```

12. assistant (2026-06-22T12:34:58.092Z)

The stub runner's ``run_predict`` writes a real trajectory CSV and returns its path. So in the join test, the stub produces a non-empty ``traj``, passing the ``if not traj`` check, and both clips get appended to ``specs``.

Now let me verify the last critical detail: that ``storage.list_uris`` / ``storage.parse_uri`` / ``storage.fetch`` work over a bare local filesystem path (the tests pass ``str(corpus)`` with no ``://`` scheme), since the whole join depends on the corpus being enumerated. Let me also confirm ``_run_startup_checks`` won't reject the empty ``{}`` config.

13. user (2026-06-22T12:35:01.157Z)

```
saved
=== run the new tests on the FIX branch (worktree checkout) ===
Preparing worktree (detached HEAD 56c1415)
HEAD is now at 56c1415 style(worker-test): drop unused os import (ruff F401)
```

14. user (2026-06-22T12:35:09.347Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\AppData\Local\Temp\wt-fix
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 29 items / 17 deselected / 12 selected
```

```

tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-22_GX020094.rallies.csv-
GX020094] PASSED [ 8%]
tests/test_worker.py::test_label_clip_name_strips_date_prefix[GX020094.rallies.csv-GX020094]
PASSED [ 16%]
tests/test_worker.py::test_label_clip_name_strips_date_prefix[GX020094_proxy.rallies.csv-
GX020094_proxy] PASSED [ 25%]
tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-
21_mahadevpura_1.rallies.csv-mahadevpura_1] PASSED [ 33%]
tests/test_worker.py::test_label_clip_name_strips_date_prefix[v1.csv-v1] PASSED [ 41%]
tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-22_GX020094.csv-GX020094]
PASSED [ 50%]
tests/test_worker.py::test_cmd_train_detector_joins_dated_labels_to_bare_videos PASSED [ 58%]
tests/test_worker.py::test_cmd_train_detector_skips_label_with_no_matching_video PASSED [ 66%]
tests/test_worker.py::test_cmd_train_detector_skips_unprobeable_and_trajless_videos PASSED [
75%]
tests/test_worker.py::test_cmd_train_detector_aborts_clean_when_fewer_than_two_videos_join
PASSED [ 83%]
tests/test_worker.py::test_cmd_train_detector_source_runs_real_extract_via_stub PASSED [ 91%]
tests/test_worker.py::test_cmd_train_detector_native_on_tracknet_fails_clean_not_traceback
PASSED [100%]

```

===== 12 passed, 17 deselected in 2.21s =====

15. assistant (2026-06-22T12:35:12.176Z)

All 12 pass on the fix branch. Now the critical test: do the regression tests actually FAIL on master? Let me apply the new tests onto a master worktree (master lacks `\_label\_clip\_name`) and run them, to confirm they genuinely guard the regression.

16. user (2026-06-22T12:35:14.985Z)

```

HEAD is now at 48f9d20 feat(nightly): make the Cloud Run scheduler path work E2E + turnkey
email/trigger (#320)
=== confirm master has NO _label_clip_name helper ===
0
=== master's cmd_train name-derivation line ===
383:         fname = storage.parse_uri(uri).name                # e.g.
GX020094_proxy.rallies.csv
384:         name = fname.replace(".rallies.csv", "").replace(".csv", "")

```

17. assistant (2026-06-22T12:35:18.638Z)

Master has no `\_label\_clip\_name` (count 0) and does the naive `name = fname.replace(...)` without stripping the date prefix. Now let me copy the new test file onto master and run the regression tests. The `\_label\_clip\_name` parametrized test should error at collection/call (AttributeError), and the detector join test should fail with rc 2.

18. user (2026-06-22T12:35:24.413Z)

```

        f.write("frame,x,y\n0,1,2\n")
    return traj
monkeypatch.setattr(wmain, "_resolve_train_detector", lambda args, config:
_FakeRunner())

out_dir = tmp_path / "out"
scratch = tmp_path / "scratch"
cfg = tmp_path / "config.json"
cfg.write_text("{}")

def fake_run(cmd, *a, **k):
    out = cmd[cmd.index("--out") + 1]
    ver = cmd[cmd.index("--version") + 1]
    bdir = os.path.join(out, ver)
    os.makedirs(bdir, exist_ok=True)

```

```

        for fn in ("weights.json", "manifest.json"):
            with open(os.path.join(bdir, fn), "w") as f:
                f.write("{}")
            return subprocess.CompletedProcess(cmd, 0)
monkeypatch.setattr(subprocess, "run", fake_run)

rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
                "--version", "0.0.6", "--config", str(cfg), "--scratch", str(scratch),
                "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
>     assert rc == 0
E     assert 2 == 0

```

```

tests\test_worker.py:350: AssertionError
----- Captured stdout call -----
[worker] trajectory source: detector
[worker] no video for '2026-06-22_badfps' under C:\Users\avidu\AppData\Local\Temp\pytest-of-
avidu\pytest-862\test_cmd_train_detector_skips_1\corpus/videos/ ? skipping.
[worker] no video for '2026-06-22_good1' under C:\Users\avidu\AppData\Local\Temp\pytest-of-
avidu\pytest-862\test_cmd_train_detector_skips_1\corpus/videos/ ? skipping.
[worker] no video for '2026-06-22_good2' under C:\Users\avidu\AppData\Local\Temp\pytest-of-
avidu\pytest-862\test_cmd_train_detector_skips_1\corpus/videos/ ? skipping.
[worker] no video for '2026-06-22_notraj' under C:\Users\avidu\AppData\Local\Temp\pytest-of-
avidu\pytest-862\test_cmd_train_detector_skips_1\corpus/videos/ ? skipping.
[worker] need >=2 videos with trajectories for leave-one-video-out; got 0.
===== short test summary info =====
FAILED
tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-22_GX020094.rallies.csv-
GX020094]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[GX020094.rallies.csv-
GX020094]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[GX020094_proxy.rallies.csv-
GX020094_proxy]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-
21_mahadevpura_1.rallies.csv-mahadevpura_1]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[v1.csv-v1]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-22_GX020094.csv-
GX020094]
FAILED tests/test_worker.py::test_cmd_train_detector_joins_dated_labels_to_bare_videos
FAILED tests/test_worker.py::test_cmd_train_detector_skips_label_with_no_matching_video
FAILED tests/test_worker.py::test_cmd_train_detector_skips_unprobeable_and_trajless_videos
===== 9 failed, 1 passed, 19 deselected in 2.94s =====

```

19. assistant (2026-06-22T12:35:32.991Z)

This is the decisive evidence. On master:

- All 6 `\_label\_clip\_name` parametrized cases FAIL (the helper doesn't exist ? `AttributeError`).
- `joins`, `no\_matching\_video`, and `unprobeable` tests FAIL with `rc 2` ? and the captured stdout shows the exact regression mechanism: master derives `name = '2026-06-22\_badfps'` (date NOT stripped), so `video\_by\_stem.get(name)` misses on every dated label ? `"no video for '2026-06-22\_good1'... ? skipping"` for ALL clips ? `got 0` ? rc 2 instead of 0.

The one that "passed" on master is the abort test (`fewer\_than\_two`), which expects rc 2 ? and on master everything aborts with rc 2, so it passes for the wrong reason. That's an important nuance: that test alone does NOT distinguish fix from bug. But it's not claimed to; it guards the *clean abort* path, and the join/skip tests carry the regression-guarding load. Let me confirm that specific test's status and verify the abort test isn't the *only* guard (it isn't ? the join test is).

Let me also verify the `fewer\_than\_two` test passed on master (expected) and double-check the join test's stdout on master to nail the mechanism.

20. user (2026-06-22T12:35:38.210Z)

```
tests/test_worker.py::test_cmd_train_detector_joins_dated_labels_to_bare_videos FAILED [ 50%]
tests/test_worker.py::test_cmd_train_detector_aborts_clean_when_fewer_than_two_videos_join
PASSED [100%]
>         assert rc == 0 # before the fix: every clip skipped -> rc 2 ("<2 videos")
E         assert 2 == 0
[worker] no video for '2026-06-21_mahadevpura_1' under C:\Users\avidu\AppData\Local\Temp\pytest-
of-avidu\pytest-865\test_cmd_train_detector_joins_0\corpus/videos/ ? skipping.
[worker] no video for '2026-06-22_GX020094' under C:\Users\avidu\AppData\Local\Temp\pytest-of-
avidu\pytest-865\test_cmd_train_detector_joins_0\corpus/videos/ ? skipping.
[worker] need >=2 videos with trajectories for leave-one-video-out; got 0.
FAILED tests/test_worker.py::test_cmd_train_detector_joins_dated_labels_to_bare_videos
```

21. assistant (2026-06-22T12:35:47.137Z)

Confirmed. Now let me verify the unprobeable/notraj test isn't tautological in a subtler way ? specifically that on the FIX branch it genuinely distinguishes the skip branches (i.e., the `{good1, good2}` assertion would catch a regression where badfps or notraj leaked through, not just an all-or-nothing pass). The fix-branch run already showed it passing with the right names. Let me also confirm the monkeypatch targets don't stub out the code under test by checking what each test patches vs. what it exercises.

Let me verify one subtle point on the unprobeable test: it monkeypatches `\_resolve\_train\_detector` to return `\_FakeRunner` ? does that stub away the date-stripping join logic? No: the join (`video\_by\_stem.get(name)`) happens in `cmd\_train` itself using `\_label\_clip\_name`, which is NOT patched. The `\_FakeRunner` only replaces trajectory production. Let me confirm `compute\_video\_id` patching and that the join still runs through the real helper in that test by checking the fix-branch stdout.

22. user (2026-06-22T12:35:51.577Z)

```
[worker] could not probe fps/width for 'badfps' (ffprobe) ? skipping (won't guess).
[worker] detector produced no trajectory for 'notraj' ? skipping.
PASSED
```

23. assistant (2026-06-22T12:35:55.787Z)

This confirms on the fix branch the skip branches fire by the RIGHT name (`badfps`, `notraj` ? date already stripped via the real `\_label\_clip\_name`), and the distinct skip messages prove the unprobeable branch (`could not probe`) and the no-trajectory branch (`produced no trajectory`) are independently exercised, not collapsed. The `\_FakeRunner` only replaces trajectory production; the date-strip join and the probe-skip logic remain real code under test.

Let me clean up the worktrees.

24. user (2026-06-22T12:35:57.361Z)

```
cleaned
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/resweep
fe34013 [docs/lovo-resweep-n13]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing
a3b209a [master]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard
5ba6955 [feat/w2-never-empty-safeguard]
```

25. assistant (2026-06-22T12:36:14.209Z)

I have concrete, executed evidence for every check. Let me summarize the verification.

****Verdict synthesis:****

1. ****`test\_label\_clip\_name\_strips\_date\_prefix`**** ? covers all meaningful cases with correct mappings: dated?bare (`2026-06-22\_GX020094.rallies.csv`?`GX020094`), legacy bare?same stem, `\_proxy` retained (proving it doesn't over-strip suffixes), underscores in `` survive (`mahadevpura\_1`), bare `.csv`, and dated+bare `.csv`. The `\_proxy`-kept and underscores-survive

cases are the non-trivial ones that guard against a too-greedy regex. All 6 FAIL on master (`AttributeError`, helper absent).

2. **test_cmd_train_detector_joins_dated_labels_to_bare_videos** ? genuinely exercises the join through `cmd\_train` end-to-end, NOT mocked away. It sets `RALLY\_DETECTOR\_IMPL=stub` and lets the REAL `\_resolve\_train\_detector` build a real stub runner that writes a real trajectory; only `\_probe\_video\_fps\_width` and `subprocess.run` (the GPU/harness boundary) are stubbed. The date-strip join and manifest assembly are real code. **Confirmed FAILS on master with `rc 2`**, stdout: `no video for '2026-06-22\_GX020094' ... skipping` ? `got 0`. Mechanism: master's `name = fname.replace(".rallies.csv","")` leaves the `2026-06-22\_` prefix, so `video\_by\_stem.get(name)` (keyed by bare stem `GX020094`) misses every dated label. Asserts both `len(specs)==2` and the exact date-stripped names `{GX020094, mahadevpura\_1}`.

3. **Skip-branch tests** assert observable outcomes, not "doesn't crash": `no\_matching\_video` asserts the surviving manifest names exclude `orphan`; `unprobeable\_and\_trajless` asserts `{good1, good2}` (badfps + notraj excluded) and the two skips fire by distinct messages/branches; `fewer\_than\_two` asserts `rc==2` AND guards that `subprocess.run` is never reached (`pytest.fail` on call).

4. **One honest caveat (nit, not blocking):**
`test\_cmd\_train\_detector\_aborts\_clean\_when\_fewer\_than\_two\_videos\_join` PASSES on master too (master aborts everything with rc 2), so that test alone does not distinguish fix from bug. But it isn't the regression guard ? it guards the legitimate clean-abort path, and the join/skip tests carry the regression load and demonstrably fail pre-fix. Monkeypatches target only the GPU/probe/harness boundary; none stub out `\_label\_clip\_name` or the join.

26. user (2026-06-22T12:36:31.919Z)

Structured output provided successfully

27. assistant (2026-06-22T12:36:39.295Z)

Verification complete. The new tests in PR #319 genuinely guard the fix and are not tautological or weak.

Key findings (all executed, not reasoned-only):

- **All 6 `\_label\_clip\_name` cases** fail on master (`AttributeError`, helper absent) and pass on the fix branch; the `\_proxy`-kept and underscores-survive cases meaningfully guard against an over-greedy regex.
- **The `joins` test** fails on master with `rc 2` via the exact regression mechanism (captured stdout: `no video for '2026-06-22\_GX020094' ... skipping` ? `got 0`). It exercises the real join through `cmd\_train` with only the GPU/probe/harness boundary stubbed ? the stub runner writes a real trajectory, so the date-strip and manifest assembly are genuinely under test.
- **The skip-branch tests** assert observable manifest names / rc and fire distinct branches by distinct messages (`could not probe`, `produced no trajectory`), not "doesn't crash."
- One honest nit: the `<2-videos abort` test passes on master too (master aborts everything with rc 2), so it alone doesn't distinguish fixed-vs-broken ? but it's not the regression guard; the join/skip tests carry that load and demonstrably fail pre-fix.

Verdict: **ok=true**, no blocking issues.